UNIVERSIDAD TÉCNICA DE AMBATO

FACULTAD DE INGENIERÍA EN SISTEMAS, ELECTRÓNICA E INDUSTRIAL

CARRERA DE SOFTWARE

ALGORITMOS Y LÓGICA DE PROGRAMACIÓN

GUÍA APE 5

Manejo de ficheros en Java: ejercicios del 1 al 10

Estudiante: Paredes Acosta Dereck Shair

Docente: Ing. José Caiza

Semestre: Primer semestre

Periodo académico: Julio – Diciembre 2026

Repositorio GitHub: https://github.com/dereckflux/T2.4Fase1APE5.-Creaci-n-Lectura-y-Escritura-de-Ficheros

Ambato – Ecuador

Noviembre 2026

Ejercicios APE 5 – Manejo de ficheros en Java

Estudiante: Paredes Acosta Dereck Shair

Universidad Técnica de Ambato – Carrera de Software

Ejercicio 1. Crear un fichero

Enunciado:

Desarrollar un programa en Java que permita crear un archivo de texto llamado curso.txt.

Explicación de la solución:

Se crea un objeto File con el nombre curso.txt y se llama a createNewFile(), que devuelve true si el archivo se creó y false si ya existía. Con getName() se muestra el nombre del archivo, y el bloque try-catch controla la IOException que puede ocurrir al crearlo.

Código fuente (Ejercicio1.java):

```
import java.io.File;
import java.io.IOException;

public class Ejercicio1 {
    public static void main(String[] args) {
        try {
            // Objeto File que representa el archivo curso.txt
            File archivo = new File("curso.txt");

            // createNewFile() devuelve true si crea el archivo y false si ya existe
            if (archivo.createNewFile()) {
                System.out.println("Archivo creado correctamente.");
            } else {
                System.out.println("El archivo ya existe.");
            }

            System.out.println("Nombre del archivo: " + archivo.getName());
        } catch (IOException e) {
            System.out.println("Se produjo un error al crear el archivo.");
        }
    }
}
```

Resultado de la ejecución:

```
--- Primera ejecución ---
Archivo creado correctamente.
Nombre del archivo: curso.txt

--- Segunda ejecución ---
El archivo ya existe.
Nombre del archivo: curso.txt
```

Ejercicio 2. Escribir información en un fichero

Enunciado:

Desarrollar un programa en Java que permita ingresar un mensaje por teclado y almacenarlo en un archivo de texto existente llamado estudiantes.txt.

Explicación de la solución:

Con Scanner se lee el mensaje que escribe el usuario. Luego se abre estudiantes.txt con FileWriter usando el parámetro true, para que el texto nuevo se agregue al final sin borrar lo anterior. El mensaje se guarda con write(), el archivo se cierra con close() y cualquier error de escritura se controla con try-catch.

Código fuente (Ejercicio2.java):

```
import java.io.FileWriter;
import java.io.IOException;
import java.util.Scanner;

public class Ejercicio2 {
    public static void main(String[] args) {
        Scanner teclado = new Scanner(System.in);

        System.out.print("Ingrese un mensaje: ");
        String mensaje = teclado.nextLine();

        try {
            // El parámetro true conserva la información anterior
            FileWriter escritor = new FileWriter("estudiantes.txt", true);
            escritor.write(mensaje + System.lineSeparator());
            escritor.close();

            System.out.println("Mensaje guardado correctamente.");
        } catch (IOException e) {
            System.out.println("Se produjo un error al escribir en el archivo.");
        }

        teclado.close();
    }
}
```

Resultado de la ejecución:

```
--- Primera ejecución ---
Ingrese un mensaje: Hola, soy estudiante de Software
Mensaje guardado correctamente.

--- Segunda ejecución ---
Ingrese un mensaje: Hoy aprendimos a manejar ficheros
Mensaje guardado correctamente.

Contenido de estudiantes.txt:
Primer Nivel - Paralelo B
Hola, soy estudiante de Software
Hoy aprendimos a manejar ficheros
```

Ejercicio 3. Eliminar un fichero

Enunciado:

Desarrollar un programa en Java que permita eliminar el archivo de texto estudiantes.txt.

Explicación de la solución:

Se crea un objeto File con el nombre estudiantes.txt y se llama a delete(), que devuelve true si el archivo se eliminó. Con un if-else se muestra un mensaje de confirmación o, en caso contrario, un mensaje indicando que no se pudo eliminar.

Código fuente (Ejercicio3.java):

```
import java.io.File;

public class Ejercicio3 {
    public static void main(String[] args) {
        // Objeto File que representa el archivo estudiantes.txt
        File archivo = new File("estudiantes.txt");

        // delete() devuelve true si elimina el archivo y false si no pudo hacerlo
        if (archivo.delete()) {
            System.out.println("Archivo eliminado correctamente.");
        } else {
            System.out.println("No se pudo eliminar el archivo. Verifique que exista.");
        }
    }
}
```

Resultado de la ejecución:

```
--- Primera ejecución ---
Archivo eliminado correctamente.

--- Segunda ejecución ---
No se pudo eliminar el archivo. Verifique que exista.
```

Ejercicio 4. Registro de estudiantes y calificaciones

Enunciado:

Desarrollar un programa en Java que permita registrar los datos de 3 estudiantes y guardar la información en un archivo llamado estudiantes.txt.

El programa debe: solicitar el nombre del estudiante; solicitar su calificación; guardar ambos datos en el archivo; hacer que cada estudiante ocupe dos líneas; utilizar Scanner, BufferedWriter y FileWriter; y cerrar correctamente el archivo.

Explicación de la solución:

Con Scanner se leen el nombre y la calificación de cada estudiante dentro de un ciclo for que se repite 3 veces. BufferedWriter, junto con FileWriter, escribe los datos en estudiantes.txt: write() guarda cada dato y newLine() pasa a la siguiente línea, por lo que cada estudiante ocupa dos líneas. El método leerNumero() vuelve a preguntar si la calificación no es válida, close() cierra el archivo y try-catch controla los errores de escritura. Como no se usa el parámetro true, cada ejecución crea el registro desde cero.

Código fuente (Ejercicio4RegistroEstudiantes.java):

```
import java.io.BufferedWriter;
import java.io.FileWriter;
import java.io.IOException;
import java.util.Scanner;

public class Ejercicio4RegistroEstudiantes {

    public static void main(String[] args) {
        Scanner teclado = new Scanner(System.in);

        try {
            FileWriter archivo = new FileWriter("estudiantes.txt");
            BufferedWriter escritor = new BufferedWriter(archivo);

            for (int i = 1; i <= 3; i++) {
                System.out.print("Nombre del estudiante " + i + ": ");
                String nombre = teclado.nextLine();
                double calificacion = leerNumero(teclado, "Calificacion: ");

                // Cada estudiante ocupa dos líneas en el archivo
                escritor.write("Estudiante: " + nombre);
                escritor.newLine();
                escritor.write("Calificacion: " + calificacion);
                escritor.newLine();
            }

            escritor.close();
            System.out.println("Datos guardados correctamente en estudiantes.txt");
        } catch (IOException e) {
            System.out.println("Se produjo un error al guardar el archivo.");
        }

        teclado.close();
    }

    // Repite la pregunta hasta recibir un número válido
    public static double leerNumero(Scanner teclado, String mensaje) {
        while (true) {
            System.out.print(mensaje);
            try {
```

```
                String texto = teclado.nextLine().trim().replace(',', '.');
                double numero = Double.parseDouble(texto);
                if (numero >= 0) {
                    return numero;
                }
            } catch (NumberFormatException e) {
                // Si no es un número, se vuelve a preguntar
            }
            System.out.println("Valor no valido. Intente de nuevo.");
        }
    }
}
```

Resultado de la ejecución:

```
Nombre del estudiante 1: Ana Torres
Calificacion: 9.5
Nombre del estudiante 2: Luis Mena
Calificacion: abc
Valor no valido. Intente de nuevo.
Calificacion: 7,25
Nombre del estudiante 3: Sofia Vega
Calificacion: 10
Datos guardados correctamente en estudiantes.txt

Contenido de estudiantes.txt:
Estudiante: Ana Torres
Calificacion: 9.5
Estudiante: Luis Mena
Calificacion: 7.25
Estudiante: Sofia Vega
Calificacion: 10.0
```

Ejercicio 5. Registro de productos con modo append

Enunciado:

Desarrollar un programa que permita registrar productos en un archivo llamado productos.txt.

El programa debe: solicitar el nombre del producto; solicitar su precio; solicitar la cantidad disponible; calcular el valor total del producto (precio × cantidad); guardar los resultados en el archivo; permitir registrar nuevos productos sin borrar los registros anteriores; y utilizar BufferedWriter y modo append.

Explicación de la solución:

Se abre productos.txt con FileWriter usando el parámetro true, que activa el modo append, y se envuelve en un BufferedWriter. Dentro de un ciclo do-while se solicitan el nombre, el precio y la cantidad, y se calcula el valor total como precio × cantidad. Cada producto se escribe en el archivo y el usuario decide si registra otro. Los métodos leerNumero() y leerEntero() validan los datos ingresados, y dinero() muestra los valores con dos decimales.

Código fuente (Ejercicio5RegistroProductos.java):

```
import java.io.BufferedWriter;
import java.io.FileWriter;
import java.io.IOException;
import java.util.Scanner;

public class Ejercicio5RegistroProductos {

    public static void main(String[] args) {
        Scanner teclado = new Scanner(System.in);

        try {
            // El parámetro true conserva los productos registrados anteriormente
            FileWriter archivo = new FileWriter("productos.txt", true);
            BufferedWriter escritor = new BufferedWriter(archivo);
            String respuesta;

            do {
                System.out.print("Nombre del producto: ");
                String nombre = teclado.nextLine();
                double precio = leerNumero(teclado, "Precio: ");
                int cantidad = leerEntero(teclado, "Cantidad disponible: ");

                double valorTotal = precio * cantidad;

                escritor.write("Producto: " + nombre);
                escritor.newLine();
                escritor.write("Precio: $" + dinero(precio));
                escritor.newLine();
                escritor.write("Cantidad: " + cantidad);
                escritor.newLine();
                escritor.write("Valor total: $" + dinero(valorTotal));
                escritor.newLine();
                escritor.write("------------------------------");
                escritor.newLine();

                System.out.println("Valor total: $" + dinero(valorTotal));
                System.out.print("Desea registrar otro producto? (s/n): ");
                respuesta = teclado.nextLine().trim();
```

```
            } while (respuesta.equalsIgnoreCase("s"));

            escritor.close();
            System.out.println("Productos guardados correctamente en productos.txt");
        } catch (IOException e) {
            System.out.println("Se produjo un error al guardar el archivo.");
        }

        teclado.close();
    }

    // Da formato de dos decimales con punto
    public static String dinero(double valor) {
        return String.format("%.2f", valor).replace(',', '.');
    }

    // Repite la pregunta hasta recibir un número válido
    public static double leerNumero(Scanner teclado, String mensaje) {
        while (true) {
            System.out.print(mensaje);
            try {
                String texto = teclado.nextLine().trim().replace(',', '.');
                double numero = Double.parseDouble(texto);
                if (numero >= 0) {
                    return numero;
                }
            } catch (NumberFormatException e) {
                // Si no es un número, se vuelve a preguntar
            }
            System.out.println("Valor no valido. Intente de nuevo.");
        }
    }

    // Repite la pregunta hasta recibir un número entero válido
    public static int leerEntero(Scanner teclado, String mensaje) {
        while (true) {
            System.out.print(mensaje);
            try {
                int numero = Integer.parseInt(teclado.nextLine().trim());
                if (numero >= 0) {
                    return numero;
                }
            } catch (NumberFormatException e) {
                // Si no es un entero, se vuelve a preguntar
            }
            System.out.println("Valor no valido. Intente de nuevo.");
        }
    }
}
```

Resultado de la ejecución:

```
--- Primera ejecución ---
Nombre del producto: Cuaderno
Precio: 2.50
Cantidad disponible: 10
Valor total: $25.00
Desea registrar otro producto? (s/n): s
Nombre del producto: Lapiz
Precio: 0.40
Cantidad disponible: 25
Valor total: $10.00
Desea registrar otro producto? (s/n): n
Productos guardados correctamente en productos.txt

--- Segunda ejecución ---
Nombre del producto: Regla
Precio: 1.25
Cantidad disponible: 8
Valor total: $10.00
Desea registrar otro producto? (s/n): n
Productos guardados correctamente en productos.txt

Contenido de productos.txt:
Producto: Cuaderno
Precio: $2.50
Cantidad: 10
Valor total: $25.00
------------------------------
Producto: Lapiz
Precio: $0.40
Cantidad: 25
Valor total: $10.00
------------------------------
Producto: Regla
Precio: $1.25
Cantidad: 8
Valor total: $10.00
------------------------------
```

Ejercicio 6. Sistema de registro de ventas

Enunciado:

Desarrollar un programa en Java que permita registrar las ventas de una tienda y guardar la información en un archivo llamado ventas.txt.

El programa debe: solicitar el nombre del producto; solicitar el precio y la cantidad vendida; calcular el subtotal de cada venta; si el subtotal es mayor o igual a $100, aplicar un descuento del 10%; mostrar el total a pagar; permitir registrar varios productos hasta que el usuario escriba fin; guardar cada venta en el archivo sin eliminar los registros anteriores; y, al finalizar, guardar el total acumulado de las ventas realizadas.

Explicación de la solución:

Un ciclo while solicita productos hasta que el usuario escribe fin. Por cada venta se calcula el subtotal como precio × cantidad y, si es mayor o igual a $100, se resta un descuento del 10%. El total a pagar se muestra en pantalla, se suma al total acumulado y se guarda en ventas.txt, que se abre en modo append para no eliminar los registros anteriores. Al terminar, se escribe el total acumulado de esa ejecución, siempre que se haya registrado al menos una venta.

Código fuente (Ejercicio6RegistroVentas.java):

```
import java.io.BufferedWriter;
import java.io.FileWriter;
import java.io.IOException;
import java.util.Scanner;

public class Ejercicio6RegistroVentas {

    public static void main(String[] args) {
        Scanner teclado = new Scanner(System.in);
        double totalAcumulado = 0;
        int ventas = 0;

        try {
            // El parámetro true conserva las ventas registradas anteriormente
            FileWriter archivo = new FileWriter("ventas.txt", true);
            BufferedWriter escritor = new BufferedWriter(archivo);

            System.out.print("Nombre del producto (fin para terminar): ");
            String producto = teclado.nextLine().trim();

            while (!producto.equalsIgnoreCase("fin")) {
                double precio = leerNumero(teclado, "Precio: ");
                int cantidad = leerEntero(teclado, "Cantidad vendida: ");

                double subtotal = precio * cantidad;
                double descuento = 0;

                // Se aplica un descuento del 10% desde $100
                if (subtotal >= 100) {
                    descuento = subtotal * 0.10;
                }

                double total = subtotal - descuento;
                totalAcumulado += total;
                ventas++;
```

```
                System.out.println("Subtotal: $" + dinero(subtotal));
                System.out.println("Descuento: $" + dinero(descuento));
                System.out.println("Total a pagar: $" + dinero(total));

                escritor.write(producto + " | Precio: $" + dinero(precio)
                        + " | Cantidad: " + cantidad
                        + " | Subtotal: $" + dinero(subtotal)
                        + " | Descuento: $" + dinero(descuento)
                        + " | Total: $" + dinero(total));
                escritor.newLine();

                System.out.print("Nombre del producto (fin para terminar): ");
                producto = teclado.nextLine().trim();
            }

            // Se guarda el total acumulado de las ventas realizadas
            if (ventas > 0) {
                escritor.write("TOTAL ACUMULADO: $" + dinero(totalAcumulado));
                escritor.newLine();
            }

            escritor.close();
            System.out.println("Total acumulado: $" + dinero(totalAcumulado));
            System.out.println("Proceso finalizado. Revise el archivo ventas.txt");
        } catch (IOException e) {
            System.out.println("Se produjo un error al guardar el archivo.");
        }

        teclado.close();
    }

    // Da formato de dos decimales con punto
    public static String dinero(double valor) {
        return String.format("%.2f", valor).replace(',', '.');
    }

    // Repite la pregunta hasta recibir un número válido
    public static double leerNumero(Scanner teclado, String mensaje) {
        while (true) {
            System.out.print(mensaje);
            try {
                String texto = teclado.nextLine().trim().replace(',', '.');
                double numero = Double.parseDouble(texto);
                if (numero >= 0) {
                    return numero;
                }
            } catch (NumberFormatException e) {
                // Si no es un número, se vuelve a preguntar
            }
            System.out.println("Valor no valido. Intente de nuevo.");
        }
    }

    // Repite la pregunta hasta recibir un número entero válido
    public static int leerEntero(Scanner teclado, String mensaje) {
        while (true) {
            System.out.print(mensaje);
            try {
                int numero = Integer.parseInt(teclado.nextLine().trim());
                if (numero >= 0) {
                    return numero;
                }
            } catch (NumberFormatException e) {
                // Si no es un entero, se vuelve a preguntar
```

```
            }
            System.out.println("Valor no valido. Intente de nuevo.");
        }
    }
}
```

Resultado de la ejecución:

```
--- Primera ejecución ---
Nombre del producto (fin para terminar): Cuaderno
Precio: 2.50
Cantidad vendida: 6
Subtotal: $15.00
Descuento: $0.00
Total a pagar: $15.00
Nombre del producto (fin para terminar): Mochila
Precio: 25
Cantidad vendida: 4
Subtotal: $100.00
Descuento: $10.00
Total a pagar: $90.00
Nombre del producto (fin para terminar): Monitor
Precio: 150.75
Cantidad vendida: 2
Subtotal: $301.50
Descuento: $30.15
Total a pagar: $271.35
Nombre del producto (fin para terminar): fin
Total acumulado: $376.35
Proceso finalizado. Revise el archivo ventas.txt

--- Segunda ejecución ---
Nombre del producto (fin para terminar): Teclado
Precio: 30
Cantidad vendida: 3
Subtotal: $90.00
Descuento: $0.00
Total a pagar: $90.00
Nombre del producto (fin para terminar): FIN
Total acumulado: $90.00
Proceso finalizado. Revise el archivo ventas.txt

Contenido de ventas.txt:
Cuaderno | Precio: $2.50 | Cantidad: 6 | Subtotal: $15.00 | Descuento: $0.00 | Total: $15.00
Mochila | Precio: $25.00 | Cantidad: 4 | Subtotal: $100.00 | Descuento: $10.00 | Total: $90.00
Monitor | Precio: $150.75 | Cantidad: 2 | Subtotal: $301.50 | Descuento: $30.15 | Total: $271.35
TOTAL ACUMULADO: $376.35
Teclado | Precio: $30.00 | Cantidad: 3 | Subtotal: $90.00 | Descuento: $0.00 | Total: $90.00
TOTAL ACUMULADO: $90.00
```

Ejercicio 7. Exploración y gestión de metadatos con File

Enunciado:

Basándote en los conceptos de rutas y abstracción de la sesión, escribe un programa en Java que utilice la clase File (perteneciente al paquete java.io) para interactuar con la ruta "datos/prueba.txt". Tu solución debe cumplir estrictamente con lo siguiente:

1. Comprobación previa: Utiliza el método adecuado (exists()) para verificar si la ruta apunta a un recurso disponible en el disco.

2. Creación controlada: Si el recurso no existe, intenta crearlo mediante el método correspondiente (createNewFile()). Como esta operación puede producir un error de E/S, debes gestionarlo obligatoriamente manejando la excepción IOException (mediante un bloque try-catch).

3. Extracción de información: Tanto si el archivo ya existía como si se acaba de crear, utiliza los métodos de consulta para mostrar por consola el último nombre de la ruta (getName()) y la ruta completa o relativa con la que se construyó el objeto (getPath()).

Explicación de la solución:

Se construye un objeto File con la ruta datos/prueba.txt y se comprueba con exists() si el archivo está disponible. Si no existe, mkdirs() prepara la carpeta datos, porque createNewFile() no crea carpetas, y luego se crea el archivo dentro de un bloque try-catch que controla la IOException. En ambos casos, getName() muestra el último nombre de la ruta y getPath() muestra la ruta con la que se construyó el objeto.

Código fuente (Ejercicio7MetadatosFile.java):

```
import java.io.File;
import java.io.IOException;

public class Ejercicio7MetadatosFile {

    public static void main(String[] args) {
        File archivo = new File("datos/prueba.txt");

        try {
            // 1. Comprobación previa: exists() indica si la ruta está disponible
            if (archivo.exists()) {
                System.out.println("El archivo ya existe.");
            } else {
                // createNewFile() no crea carpetas, por eso se prepara "datos"
                archivo.getParentFile().mkdirs();

                // 2. Creación controlada
                if (archivo.createNewFile()) {
                    System.out.println("Archivo creado correctamente.");
                }
            }

            // 3. Extracción de información
            System.out.println("Nombre: " + archivo.getName());
            System.out.println("Ruta: " + archivo.getPath());
```

```
        } catch (IOException e) {
            System.out.println("Se produjo un error al crear el archivo.");
        }
    }
}
```

Resultado de la ejecución:

```
--- Primera ejecución ---
Archivo creado correctamente.
Nombre: prueba.txt
Ruta: datos/prueba.txt

--- Segunda ejecución ---
El archivo ya existe.
Nombre: prueba.txt
Ruta: datos/prueba.txt
```

Ejercicio 8. Escritura estructurada por líneas con BufferedWriter

Enunciado:

Tomando como referencia el flujo de datos desde la memoria RAM hasta el archivo en disco, desarrolla un programa en Java que cree un fichero de texto llamado "materias.txt". El programa debe realizar las siguientes acciones:

1. Configurar un flujo de salida utilizando un objeto BufferedWriter enlazado con un FileWriter.

2. Escribir un listado de cinco asignaturas de tu elección en el archivo.

3. Emplear el método portable newLine() para asegurar que cada asignatura quede guardada estrictamente en una línea independiente.

4. Liberar los recursos de forma correcta invocando el método de cierre (close()) y controlar cualquier error potencial de escritura mediante una excepción de tipo IOException.

5. Verificación: Comprueba físicamente en la carpeta de tu proyecto que el archivo se generó correctamente y que mantiene una única asignatura por línea.

Explicación de la solución:

Las cinco asignaturas se guardan en un arreglo. Se configura el flujo de salida enlazando un BufferedWriter con un FileWriter y, con un ciclo for, write() escribe cada asignatura y newLine() agrega el salto de línea adecuado para el sistema. Al terminar, close() libera los recursos y el bloque try-catch controla los errores de escritura.

Código fuente (Ejercicio8EscrituraMaterias.java):

```
import java.io.BufferedWriter;
import java.io.FileWriter;
import java.io.IOException;

public class Ejercicio8EscrituraMaterias {

    public static void main(String[] args) {
        String[] asignaturas = {
            "Algoritmos",
            "Sistemas Operativos",
            "Base de Datos",
            "Redes de Computadoras",
            "Estructura de Datos"
        };

        try {
            // 1. Flujo de salida: BufferedWriter enlazado con FileWriter
            FileWriter archivo = new FileWriter("materias.txt");
            BufferedWriter escritor = new BufferedWriter(archivo);

            // 2 y 3. Cada asignatura queda en una línea independiente
            for (int i = 0; i < asignaturas.length; i++) {
                escritor.write(asignaturas[i]);
                escritor.newLine();
            }

            // 4. Liberación de recursos
            escritor.close();
```

```
            System.out.println("Archivo materias.txt creado con 5 asignaturas.");
        } catch (IOException e) {
            System.out.println("Se produjo un error al escribir en el archivo.");
        }
    }
}
```

Resultado de la ejecución:

```
Archivo materias.txt creado con 5 asignaturas.

Contenido de materias.txt:
Algoritmos
Sistemas Operativos
Base de Datos
Redes de Computadoras
Estructura de Datos
```

Ejercicio 9. Control del modo de apertura (append vs sobrescribir)

Enunciado:

A partir del archivo "materias.txt" generado en el ejercicio anterior, escribe un nuevo programa en Java que demuestre el comportamiento del segundo parámetro en el constructor de FileWriter:

1. Diseña el código de manera que abra el fichero existente configurando el FileWriter en modo de adición (append = true).

2. Añade dos nuevas asignaturas al final del archivo utilizando nuevamente newLine() para no romper el formato de líneas.

3. Asegúrate de cerrar el flujo con .close().

4. Comprobación: Verifica que el contenido previo no se haya sobrescribido y que ahora el archivo conserve las cinco asignaturas originales junto con las dos nuevas incorporadas al final.

Explicación de la solución:

Se abre materias.txt con FileWriter y el segundo parámetro en true, que activa el modo de adición. Con BufferedWriter se escriben dos asignaturas nuevas, usando newLine() para mantener el formato de una por línea, y close() cierra el flujo. Si el parámetro fuera false o no se indicara, el archivo se sobrescribiría y se perderían las cinco asignaturas originales.

Código fuente (Ejercicio9ModoAppend.java):

```
import java.io.BufferedWriter;
import java.io.FileWriter;
import java.io.IOException;

public class Ejercicio9ModoAppend {

    public static void main(String[] args) {
        try {
            // 1. El segundo parámetro en true activa el modo de adición
            FileWriter archivo = new FileWriter("materias.txt", true);
            BufferedWriter escritor = new BufferedWriter(archivo);

            // 2. Las nuevas asignaturas se agregan al final
            escritor.write("Desarrollo Web");
            escritor.newLine();
            escritor.write("Ciberseguridad");
            escritor.newLine();

            // 3. Cierre del flujo
            escritor.close();
            System.out.println("Se agregaron 2 asignaturas a materias.txt.");
        } catch (IOException e) {
            System.out.println("Se produjo un error al escribir en el archivo.");
        }
    }
}
```

Resultado de la ejecución:

```
Se agregaron 2 asignaturas a materias.txt.

Contenido de materias.txt:
Algoritmos
Sistemas Operativos
Base de Datos
Redes de Computadoras
Estructura de Datos
Desarrollo Web
Ciberseguridad
```

Ejercicio 10. Ciclo de vida completo y borrado seguro

Enunciado:

Diseña un script completo en Java que emule el recorrido guiado de gestión de ficheros visto en clase:

1. Construye un objeto File que represente la ruta "datos/temporal.txt".

2. Consulta el estado del archivo mediante exists() y, si no se encuentra disponible, créalo de forma segura controlando la excepción IOException.

3. Implementa una fase de validación en la que declares una variable de control booleana (por ejemplo, boolean confirmarBorrado = true;) para simular la solicitud de confirmación por parte del usuario antes de eliminar datos importantes.

4. Utiliza el método delete() exclusivamente si dicha confirmación es verdadera, imprimiendo por consola un mensaje que refleje el resultado booleano de la operación de eliminación.

Explicación de la solución:

Se construye un objeto File con la ruta datos/temporal.txt. Con exists() se consulta su estado y, si no está disponible, se prepara la carpeta datos y se crea el archivo controlando la IOException. La variable booleana confirmarBorrado simula la confirmación del usuario: delete() solo se ejecuta si es true, y su resultado booleano se muestra por consola. Si es false, el archivo se conserva.

Código fuente (Ejercicio10CicloVida.java):

```
import java.io.File;
import java.io.IOException;

public class Ejercicio10CicloVida {

    public static void main(String[] args) {
        // 1. Objeto File que representa la ruta
        File archivo = new File("datos/temporal.txt");

        // 2. Consulta del estado y creación segura
        try {
            if (archivo.exists()) {
                System.out.println("El archivo ya existe.");
            } else {
                // createNewFile() no crea carpetas, por eso se prepara "datos"
                archivo.getParentFile().mkdirs();

                if (archivo.createNewFile()) {
                    System.out.println("Archivo creado: " + archivo.getPath());
                }
            }
        } catch (IOException e) {
            System.out.println("Se produjo un error al crear el archivo.");
        }

        // 3. Variable de control que simula la confirmación del usuario
        boolean confirmarBorrado = true;

        // 4. delete() solo se ejecuta si la confirmación es verdadera
        if (confirmarBorrado) {
```

```
            boolean eliminado = archivo.delete();
            System.out.println("Resultado de delete(): " + eliminado);
        } else {
            System.out.println("Borrado cancelado. El archivo se conserva.");
        }
    }
}
```

Resultado de la ejecución:

```
Con confirmarBorrado = true:
Archivo creado: datos/temporal.txt
Resultado de delete(): true

Con confirmarBorrado = false:
Archivo creado: datos/temporal.txt
Borrado cancelado. El archivo se conserva.
```

Conclusiones

1. La clase File permite representar un archivo y gestionarlo: exists() consulta su estado, createNewFile() lo crea, getName() y getPath() muestran su información y delete() lo elimina.
2. BufferedWriter, junto con FileWriter, permite escribir la información de forma ordenada línea por línea con write() y newLine(), y es importante cerrarlo con close() para que los datos se guarden.
3. El segundo parámetro de FileWriter define el modo de apertura: en true agrega la información al final y conserva el contenido anterior, mientras que sin él el archivo se sobrescribe.
4. El uso de try-catch con IOException, junto con validaciones y confirmaciones como confirmarBorrado, evita que el programa se detenga de forma inesperada o que se eliminen datos sin autorización.